第 2 章 基本数据管理

2.1 实验介绍

作为关系数据库，openGauss 的主要功能是进行关系数据的管理，包括数据定义、数据查询、数据更新等基本数据管理操作。数据定义是进行数据管理的第一步工作，其要创建数据库和定义所有数据表，作为装载具体数据的框架结构；数据查询是要在数据库中查找满足各种条件要求的数据，是数据库最重要的功能之一，需要通过大量实验操作和练习才能掌握编写查询的技巧；同时，数据更新是要实现数据添加、修改和删除操作。在本章中，我们将在 openGauss 中使用关系数据库语言 SQL 创建数据库和数据表，完成基本查询、连接查询、子查询和集合查询等各类数据查询，进行数据更新和数据表结构的修改。本章内容是对 openGauss 的基本数据管理功能进行实践。

2.2 实验目的

1. 掌握在 openGauss 中使用 SQL 创建数据库和数据表。
2. 掌握在 openGauss 中使用 SQL 进行基本查询。
3. 掌握在 openGauss 中使用 SQL 进行连接查询。
4. 掌握在 openGauss 中使用 SQL 进行子查询。
5. 掌握在 openGauss 中使用 SQL 进行集合查询。
6. 掌握在 openGauss 中使用 SQL 进行数据更新，包括添加、修改、删除数据。
7. 掌握在 openGauss 中使用 SQL 修改表结构。

2.3 实验原理

2.3.1 关系数据库语言 SQL

SQL 是关系数据库的标准语言，以统一的方式管理和操作关系数据。SQL 的主要特点包括：

(1) 功能全面

虽然 SQL 叫做“结构化查询语言”，但 SQL 的功能包括数据定义、数据查询、数据操纵和数据控制等。SQL 功能全面，语法风格统一。使用 SQL 可以独立完成数据库生命周期中的全部活动。

(2) 声明式语言

SQL 是一种声明式语言，因为使用 SQL 语句表达要做的事情时，用户只需要声明想做什么，而无需具体指出怎么完成这件事情的详细步骤。如何执行 SQL 语句的详细步骤交由 DBMS 内部自动完成。从用户的角度看，SQL 只关心“做什么”，不在乎“怎么做”。

(3) 面向集合

关系实际上就是元组的集合。将元组集合作为 SQL 语句的输入，查询语句的输出结果

是满足条件的元组集合，更新语句所影响的范围也是一个元组集合。SQL 以面向集合的操作方式对数据进行处理。

(4) 使用方式灵活

SQL 既可以用作交互式语言，又可以用作嵌入式语言。作为交互式语言，用户可以直接在数据库客户端程序中输入 SQL 命令，查看执行结果。作为嵌入式语言，SQL 可以通过各种方式嵌入到主流编程语言中，供程序开发人员使用。

(5) 易学易用

SQL 经过了精心的设计，语言十分简洁。SQL 作为进行数据管理的计算机语言，其语法与英语类似，非常容易学习和使用。

SQL 的核心功能由 4 个部分组成：

(1) 数据定义语言（Data Definition Language，简称 DDL），用于定义数据库对象。数据定义语言还可以修改或删除数据库对象的定义。

(2) 数据查询语言（Data Query Language，简称 DQL），用于根据指定条件查询数据。由于查询操作是用户最常使用的功能，数据查询语言是 SQL 最重要的组成部分。

(3) 数据操纵语言（Data Manipulation Language，简称 DML），用于更新指定的数据，包括插入新数据、修改已有数据和删除数据。

(4) 数据控制语言（Data Control Language，简称 DCL），用于管理对于数据的访问授权。控制数据库中的用户对数据项的查看或修改。

表 2.1 列出了组成 SQL 的上述 4 项功能及对应语句的关键字动词。

表 2.1 SQL 组成部分与对应语句的关键字动词

SQL 组成部分	别名
数据定义语言（DDL）	CREATE, ALTER, DROP
数据查询语言（DQL）	SELECT
数据操纵语言（DML）	INSERT, UPDATE, DELETE
数据控制语言（DCL）	GRANT, REVOKE

2.3.2 数据定义

从本章开始，我们引入“学生-课程”数据库（命名为 dbsc）作为实践 openGauss 数据库功能的示例。dbsc 数据库描述了学校中学生选修课程的情景，记录学生数据、课程数据和选课数据。**错误!未找到引用源。**是“学生-课程”数据库的 ER 图，这是一个典型的多对多关系，一个学生可以选修多门课程，一门课程也可以由多个学生来选修。

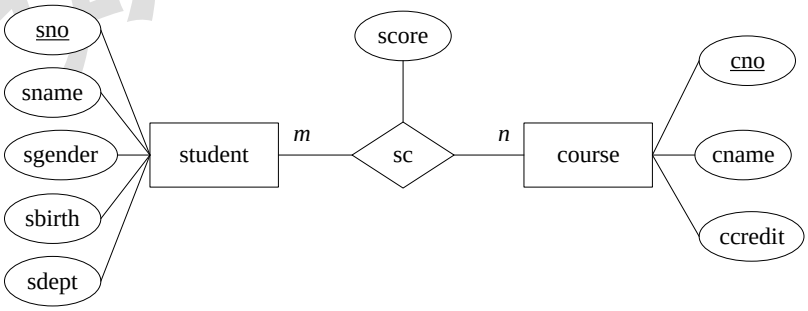

图 2.1 “学生-选课”示例数据库 ER 图

dbsc 数据库包括 3 个关系表，它们分别是：

- 学生表：student (sno, sname, sgender, sbirth, sdept)；

- 课程表：course (cno, cname, ccredit);
- 选课表：sc (sno, cno, score)。

带有下划线的字段表示主键。表 2.2、表 2.3 和表 2.4 分别列出了学生表 student、课程表 course 和选课表 sc 中的示例数据。

表 2.2 student 表示例数据

sno 学号	sname 姓名	sgender 性别	sbirth 出生日期	sdept 系别
08001	张三	男	1988-02-19	CS
08002	李四	女	1989-01-09	CS
08003	王五	女	1990-12-08	CE
08004	赵六	男	1989-08-30	IS

表 2.3 course 表示例数据

cno 课号	cname 课程名	ccredit 学分
1	高等数学	4
2	数据库原理	5
3	操作系统	3
4	信息系统	4

表 2.4 sc 表示例数据

sno 学号	cno 课号	grade 成绩
08001	1	92
08001	2	85
08001	3	88
08002	2	90
08002	3	80

在关系数据库中，关系数据模型由相互关联的若干表构成。表的创建、修改和删除操作是 SQL 中最常用的数据定义语句。

1. 创建表

在 SQL 中，使用 CREATE TABLE 语句定义基本表，语法格式为：

```
CREATE TABLE <表名>
(
  <列名 1> <数据类型> [列级完整性约束],
  <列名 2> <数据类型> [列级完整性约束],
  …
  [表级完整性约束 1],
  [表级完整性约束 2]
  …
);
```

创建表时需要指定表名，表中每个列的定义，以及完整性约束情况。可以将创建表想象为搭建用于容纳数据的结构框架或容器。描述这个框架本身的数据称为元数据（metadata），即关于数据的数据，例如，表名、列名和列的数据类型等。

2. 修改表

如果用户的需求发生了变化，表的结构要进行相应的调整。对于已经装载了大量数据的表，或者被其他数据库对象引用（例如参照完整性）的表，调整表结构不宜采取删除重建的办法。SQL 提供了 ALTER TABLE 语句用于修改基本表，语法格式为：

```
ALTER TABLE <表名>
{ ADD <列名> <数据类型> [完整性约束]
```

```
| DROP <列名> [ RESTRICT | CASCADE ]
| ALTER COLUMN <列名> <数据类型>
};
```

要修改的表由<表名>指定。ADD 子句用于向基本表中添加新列，描述新列的语法格式与 CREATE TABLE 中定义列的格式相同。DROP 子句用于删除表中已有的列。ALTER COLUMN 子句用于修改已有列的定义。注意，ALTER TABLE 修改的是表本身的结构（即表的定义），而不是表中装载的数据。

3. 删除表

如果不再需要某个基本表，可以使用 DROP TABLE 语句将其删除，该语句的语法格式为：

```
DROP TABLE <表名> [ RESTRICT | CASCADE ];
```

该语句会将基本表连同其中存储的数据一并删除。如果只想清空表中的数据，而不想删除表本身，不要使用 DROP TABLE 语句。

如果指定了 CASCADE 方式，则进行级联删除。在删除基本表时，依赖于该表的其他数据库对象会被一起删除。

如果指定了 RESTRICT 方式，则进行限制删除。要删除的表不能被其他数据库对象所引用，否则 DBMS 会拒绝删除该基本表。

如果不指定删除方式，默认删除方式是 RESTRICT。

4. 数据类型

SQL 数据类型规定了一个属性的取值范围。数据类型直接对应于关系模型中域的概念。关系中的每个属性必须来自一个域。表中每个列的取值必须符合定义表时指定的数据类型。表 2.5 列出了 SQL 提供的主要数据类型。

表 2.5 SQL 的主要数据类型

数据类型	描述
字符串型	
CHAR(n)	长度为 n 的定长字符串
VARCHAR(n)	最大长度为 n 的变长字符串
数值型	
INT	长整数（也可写作 INTEGER），通常为 4 字节
SMALLINT	短整数，通常为 2 字节
REAL	单精度浮点数
DOUBLE PRECISION	双精度浮点数
NUMERIC(p, s)	定点数，由 p 位数字组成（不包括符号和小数点），小数点后面有 s 位数字（也可写作 DECIMAL(p, s)）。p 称为精度（precision），s 称为刻度（scale）。例如，23.5141 的精度为 6，刻度为 4，即 p=6，s=4。不等式 0<=s<=p 成立。
位串型	
BIT(n)	长度为 n 的定长二进制位串
BIT VARYING(n)	最大长度为 n 的变长二进制位串
日期时间型	
DATE	日期，包含年、月、日，格式为 YYYY-MM-DD
TIME	时间，包含一日的时、分、秒，格式为 HH:MM:SS

2.3.3 数据查询

数据查询是 SQL 的核心。声明式的 SELECT 语句提供了全面的 SQL 数据查询功能。SELECT 语句的查询返回结果叫做结果集（result set）。可以将结果集想象为内存中的一张新表，SELECT 语句描述如何生成该表的行和列。

在 SQL 中，SELECT 语句的语法格式为：

```
SELECT [ ALL | DISTINCT ] <列表达式>, …
FROM <表名>, …
[WHERE <条件表达式>]
[GROUP BY <列名>, … [HAVING <条件表达式>] ]
[ORDER BY <列名> [ASC | DESC], … ];
```

SELECT 后面的<列表达式>指定结果表将包含哪些列，FROM 后面的<表名>指定数据来自哪里，WHERE 后面的<条件表达式>指定结果表中每行数据要满足的条件。SELECT 语句的总体执行步骤是：

(1) 根据 WHERE 子句的条件，
(2) 从 FROM 子句指定的表中找出满足条件的元组，
(3) 再按 SELECT 子句选出指定的列，
(4) 形成结果表。
(5) 如果有 GROUP BY 子句，
  (a) 则结果按<列名>的值分组，属性值相等的元组分为一组。
  (b) 通常会在每组中作用聚合函数。
  (c) 如果带有 HAVING 子句，则只返回满足条件的组。
(6) 如果有 ORDER BY 子句，
  (a) 则结果表还要按<列名>的值的升序或降序排序。

SELECT 语句既可以完成简单的单表查询，也可以完成复杂的连接查询和子查询。

通过 WHERE 子句可以选择表中满足指定条件的行。WHERE 子句中主要的查询条件如表 2.6 所示。WHERE 子句中指定的<条件表达式>也称为谓词。

表 2.6 WHERE 子句的查询条件

查询条件	谓词
比较	=，>，<，>=，<=，<>
范围	BETWEEN…AND…，NOT BETWEEN…AND…
集合	IN，NOT IN
字符匹配	LIKE，NOT LIKE
空值	IS NULL，IS NOT NULL
逻辑运算	AND，OR，NOT

2.3.4 数据更新

SQL 中的数据操纵语言（DML）用于更新指定的数据，包括插入新数据（INSERT 语句）、修改已有数据（UPDATE 语句）和删除数据（DELETE 语句）这三类操作。

1. 插入数据

INSERT 语句用于向表中插入新的行，有两种形式：一种是插入一个元组，另一种是插入子查询的结果。

(1) 插入元组

向表中插入一个元组的 INSERT 语句格式为：

```
INSERT INTO <表名> [(<列 1>, <列 2>, …)]
VALUES (<值 1>, <值 2>, …)
```

其功能是向<表名>指定的表中插入一行新数据。<表名>后面是属性列的列表，属性列的顺序可以不同于表中列的顺序。VALUES 后面是值的列表，指定新元组各列的取值。<列 1>的取值为<值 1>，<列 2>的取值为<值 2>，以此类推。

(2) 插入子查询结果

INSERT 语句中可以嵌套 SELECT 子查询，用于将其他表或视图中满足要求的数据行批量插入指定表。语法格式为：

```
INSERT INTO <表名> [(<列 1>, <列 2>, …)]
子查询
```

2. 修改数据

修改数据库中已有的数据称为更新操作。使用 UPDATE 语句可以更新表中满足条件的行。可以做到单独更新一列数据，而不影响其他列。UPDATE 语句的格式为：

```
UPDATE <表名>
SET <列名 1> = <新值> [, <列名 2> = <新值> …]
[WHERE <条件>]
```

要执行一个更新操作，需要指明以下三项信息：

(a) 表名和要更新的列；

(b) 该列的新值；

(c) 要更新哪些行。

3. 删除数据

DELETE 语句用于删除表中的行。语法格式为：

```
DELETE FROM <表名>
[WHERE <条件>]
```

DELETE 语句的功能是从指定表中删除满足 WHERE 子句条件的行。如果省略 WHERE 子句，表示删除表中的全部行。DELETE 语句删除的是表中的数据，而不是表的定义，表的定义仍在数据库的数据字典中。要想删除表的定义，应使用 DROP 语句删除数据库对象。

2.3.5 gsql 元命令

gsql 客户端提供若干元命令，元命令可帮助管理员查看数据库对象的信息、格式化 SQL 输出结果、以及连接到新的数据库等。表 2.7 给出了 gsql 提供的部分元命令的功能说明。

表 2.7 gsql 提供的部分元命令功能说明

元命令	功能说明
\h [NAME]	给出指定 SQL 语句的语法帮助。

\q	退出 gsql 程序。
\dt[S+] [PATTERN]	列出表 PATTERN 信息，+会列出更详细信息，S 包括系统对象。
\l	列出所有数据库的信息。
\c [NAME]	连接到一个新的数据库 NAME。
\timing [on|off]	以毫秒为单位显示每条 SQL 语句的执行时间。
\set [NAME [VALUE]]	设置内部变量 NAME 的值为 VALUE。
\unset NAME	删除变量 NAME。

2.4 实验步骤

2.4.1 创建数据库与数据表

首先，创建 dbsc 数据库和其中的数据表。

1. 以 omm 用户身份登录：

```
[omm@eduog ~]$
```

2. 启动数据库服务器：

```
[omm@eduog ~]$ gs_ctl start -D $GAUSSHOME/data/single_node -Z single_node
```

3. 连接数据库服务器：

```
[omm@eduog ~]$ gsql -d postgres -p 26000 -r
```

4. 创建“学生-课程”数据库 dbsc：

```
openGauss=# CREATE DATABASE dbsc;
CREATE DATABASE
```

5. 列出所有数据库：

```
openGauss=# \l
                                  List of databases
   Name    |  Owner   | Encoding |   Collate   |    Ctype    | Access privileges
-----------+----------+----------+-------------+-------------+-------------------
 dbsc      | omm      | UTF8     | en_US.UTF-8 | en_US.UTF-8 |
 finance   | omm      | UTF8     | en_US.UTF-8 | en_US.UTF-8 |
 postgres  | omm      | UTF8     | en_US.UTF-8 | en_US.UTF-8 |
 school    | omm      | UTF8     | en_US.UTF-8 | en_US.UTF-8 |
 template0 | omm      | UTF8     | en_US.UTF-8 | en_US.UTF-8 | =c/omm           +
           |          |          |             |             | omm=CTc/omm
 template1 | omm      | UTF8     | en_US.UTF-8 | en_US.UTF-8 | =c/omm           +
           |          |          |             |             | omm=CTc/omm
 testdb    | testuser | UTF8     | en_US.UTF-8 | en_US.UTF-8 |
(7 rows)
```

可以看到包括刚创建的 dbsc 数据库，所有者为 omm 用户。

6. 连接 dbsc 数据库：

创建数据库并不会改变当前客户端连接的数据库。需要使用\c 命令指定连接某个数据库：

```
openGauss=# \c dbsc
Non-SSL connection (SSL connection is recommended when requiring high-security)
```

```
You are now connected to database "dbsc" as user "omm".
dbsc=#
```

这样，后续创建数据表的操作才能在 dbsc 数据库中进行，否则，将在默认连接的系统数据库 postgres 中创建数据表。

7. 创建学生数据表 student：

   在 gsql 中输入以下 CREATE TABLE 语句：

```
CREATE TABLE student
(
     sno         CHAR(5),
     sname       VARCHAR(10) NOT NULL,
     sgender     CHAR(4)     NOT NULL,
     sbirth      DATE        NOT NULL,
     sdept       CHAR(2),
     CONSTRAINT pk_student PRIMARY KEY(sno)
);
```

   返回如下信息，表明数据表创建成功。

```
NOTICE:  CREATE TABLE / PRIMARY KEY will create implicit index "pk_student" for table
"student"
CREATE TABLE
```

   NOTICE 一行的意思是说 CREATE TABLE 语句会自动创建主键列上的索引。

8. 创建课程数据表 course：

```
CREATE TABLE course
(
    cno         SMALLINT,
    cname       VARCHAR(20) NOT NULL,
    ccredit     SMALLINT    NOT NULL,
    CONSTRAINT pk_course PRIMARY KEY(cno)
);
```

9. 创建选课数据表 sc：

```
CREATE TABLE sc
(
    sno      CHAR(5),
    cno      SMALLINT,
    score    SMALLINT,
    CONSTRAINT pk_sc PRIMARY KEY(sno, cno),
    CONSTRAINT fk_student FOREIGN KEY(sno) REFERENCES student(sno),
    CONSTRAINT fk_course FOREIGN KEY(cno) REFERENCES course(cno)
);
```

10. 可以使用 gsql 元命令查看当前数据库的相关信息。

    例如，使用元命令\d 查看数据表的结构。查看 student 表结构的命令如下：

```
dbsc=# \d student
                Table "public.student"
 Column  |              Type              | Modifiers
---------+--------------------------------+-----------
```

```
 sno     | character(5)                   | not null
 sname   | character varying(10)          | not null
 sgender | character(4)                   | not null
 sbirth  | timestamp(0) without time zone | not null
 sdept   | character(2)                   |
Indexes:
    "pk_student" PRIMARY KEY, btree (sno) TABLESPACE pg_default
Referenced by:
    TABLE "sc" CONSTRAINT "fk_student" FOREIGN KEY (sno) REFERENCES student(sno)
```

例如，使用元命令\dt 列出所有数据表。列出 dbsc 数据库中所有表的信息：

```
dbsc=# \dt
                          List of relations
 Schema |  Name   | Type  | Owner |             Storage
--------+---------+-------+-------+---------------------------------
 public | course  | table | omm   | {orientation=row,compression=no}
 public | sc      | table | omm   | {orientation=row,compression=no}
 public | student | table | omm   | {orientation=row,compression=no}
(3 rows)
```

Schema 列为表所在模式，Name 列为表名，Type 列为数据库对象类型，Owner 列为表所属的用户，Storage 列中是有关表的存储信息。

11. 向数据表中插入数据：

```
INSERT INTO student VALUES ('22001', '张三', '男', '2002-05-19', 'CS');
INSERT INTO student VALUES ('22002', '李四', '女', '2002-01-09', 'CS');
INSERT INTO student VALUES ('22003', '王五', '女', '2001-12-08', 'CE');
INSERT INTO student VALUES ('22004', '赵六', '男', '2001-08-30', 'IS');

INSERT INTO course VALUES (1, '高等数学', 4);
INSERT INTO course VALUES (2, '数据库系统', 5);
INSERT INTO course VALUES (3, '操作系统', 3);
INSERT INTO course VALUES (4, '信息系统', 4);

INSERT INTO sc VALUES ('22001', 1, 96);
INSERT INTO sc VALUES ('22001', 2, 85);
INSERT INTO sc VALUES ('22001', 3, 88);
INSERT INTO sc VALUES ('22002', 2, 90);
INSERT INTO sc VALUES ('22002', 3, 80);
```

12. 查看表中的数据：

列出 student 表中的数据：

```
dbsc=# SELECT * FROM student;
  sno  | sname | sgender |       sbirth        | sdept
-------+-------+---------+---------------------+-------
 22001 | 张三  | 男      | 2002-05-19 00:00:00 | CS
 22002 | 李四  | 女      | 2002-01-09 00:00:00 | CS
 22003 | 王五  | 女      | 2001-12-08 00:00:00 | CE
```

```
 22004 | 赵六  | 男      | 2001-08-30 00:00:00 | IS
```

(4 rows)列出 course 表中的数据:

```
dbsc=# SELECT * FROM course;
 cno |   cname    | ccredit
-----+------------+---------
   1 | 高等数学   |       4
   2 | 数据库系统 |       5
   3 | 操作系统   |       3
   4 | 信息系统   |       4
(4 rows)
```

列出 sc 表中的数据:

```
dbsc=# SELECT * FROM sc;
  sno  | cno | score
-------+-----+-------
 22001 |   1 | 96
 22001 |   2 | 85
 22001 |   3 | 88
 22002 |   2 | 90
 22002 |   3 | 80
(5 rows)
```

2.4.2 查询数据——基本查询

本节开始以“学生-课程”数据库上的查询任务为例，演练使用 SQL 中的 SELECT 语句进行数据查询操作。

基本查询操作一般仅涉及到一个表，因此也称为单表查询。

1. 选择表中的列。

由 SELECT 指定列名，选择表中的全部列或部分列，这对应于关系代数中的投影运算。

(1) 查询指定列

如果用户只对表中的一部分属性列感兴趣，可以在 SELECT 后面的<列表达式>中指定要查询的属性列。

【查询 1】查询全体学生的姓名与性别。

```
SELECT sname, sgender
FROM student;
sname | sgender
-------+---------
 张三  | 男
 李四  | 女
 王五  | 女
 赵六  | 男
(4 rows)
```

【查询 2】查询全体学生的姓名、学号和所在系。

```
SELECT sname, sno, sdept
```

```
FROM student;
sname |  sno  | sdept
-------+-------+-------
 张三  | 22001 | CS
 李四  | 22002 | CS
 王五  | 22003 | CE
 赵六  | 22004 | IS
(4 rows)
```

(2) 查询全部列

要获取表中的全部属性列，可以将 SELECT 后面的<列表达式>指定为*字符。

【查询 3】查询全体学生的所有信息。

```
SELECT *
FROM student;
  sno  | sname | sgender |       sbirth        | sdept
-------+-------+---------+---------------------+-------
 22001 | 张三  | 男      | 2002-05-19 00:00:00 | CS
 22002 | 李四  | 女      | 2002-01-09 00:00:00 | CS
 22003 | 王五  | 女      | 2001-12-08 00:00:00 | CE
 22004 | 赵六  | 男      | 2001-08-30 00:00:00 | IS
(4 rows)
```

该查询等价于在 SELECT 后面列出表的所有列：

```
SELECT sno, sname, sgender, sbirth, sdept
FROM student;
```

(3) 对查询列进行计算

SELECT 后面的<列表达式>除了是表中的属性列之外，还可以是任意复杂的表达式。

【查询 4】查询全体学生的姓名和年龄。

```
SELECT sname AS 姓名, AGE(CURRENT_DATE, sbirth) AS 年龄
FROM student;
姓名 |          年龄
------+-------------------------
 张三 | 21 years 9 mons 6 days
 李四 | 22 years 1 mon 16 days
 王五 | 22 years 2 mons 17 days
 赵六 | 22 years 5 mons 26 days
(4 rows)
```

该查询使用了 openGauss 提供的时间日期函数。CURRENT_DATE 函数返回当前日期；AGE 函数计算年龄，即将两个日期的差值以多少年月日的形式返回。使用 AS 关键字指定经过计算的列的别名。

如果要计算周岁，即只返回年份部分，可以使用 EXTRACT 函数对 AGE 函数返回值提取年份部分，查询如下：

```
SELECT sname AS 姓名, EXTRACT('YEAR' FROM AGE(CURRENT_DATE, sbirth)) AS 年龄
FROM student;
姓名 | 年龄
------+------
```

```
 张三 |   21
 李四 |   22
 王五 |   22
 赵六 |   22
(4 rows)
```

(4) 消除重复行

查询表中的指定列后，结果可能出现重复的行。例如：

【查询 5】查询选修了课程的学生的学号。

```
SELECT sno FROM sc;
  sno
-------
 22001
 22001
 22001
 22002
 22002
(5 rows)
```

加上 DISTINCT 关键字可消除重复的行。

```
SELECT DISTINCT sno FROM sc;
  sno
-------
 22002
 22001
(2 rows)
```

2. 选择表中的行。

通过 WHERE 子句可以选择表中满足指定条件的行，这对应于关系代数中的选择运算。

(1) 比较大小

比较大小操作使用比较运算符实现，包括：

=（等于）、>（大于）、<（小于）、>=（大于等于）、<=（小于等于）和<>（不等于）。

【查询 6】查询计算机科学系（CS）全体学生的名单。

```
SELECT sname
FROM student
WHERE sdept = 'CS';
 sname
-------
 张三
 李四
(2 rows)
```

WHERE 子句就像是一个过滤器，它将不满足条件的行排除在结果表之外，而只允许满足条件的行返回。

【查询 7】查询有不及格课程成绩的学生学号。

```
SELECT DISTINCT sno
```

```
FROM sc
WHERE score < 60;
sno
-----
(0 rows)
```

查询结果显示没有有不及格课程成绩的学生。

(2) 确定范围

使用谓词 BETWEEN…AND…（或 NOT BETWEEN…AND…）可以查找属性值在（或不在）指定范围内的元组，其中 BETWEEN 后面是范围的下限，AND 后面是范围的上限。

a BETWEEN *x* AND *y* 等价于 *a* >= *x* AND *a* <= *y*

a NOT BETWEEN *x* AND *y* 等价于 *a* < *x* OR *a* > *y*

【查询 8】查询 2 号课程成绩在 80 到 89 之间的学生学号。

```
SELECT sno
FROM SC
WHERE cno = 2 AND score BETWEEN 80 AND 89;
  sno
-------
 22001
(1 row)
```

在 BETWEEN…AND…中范围包括上下限（即上述查询条件包括 80 和 89），而在 NOT BETWEEN…AND…中范围不包括上下限，这两个谓词确定的范围是互补的。

【查询 9】查询 2 号课程成绩不在 80 到 89 之间的学生学号。

```
SELECT sno
FROM SC
WHERE cno = 2 AND score NOT BETWEEN 80 AND 89;
  sno
-------
 22002
(1 row)
```

(3) 确定集合

谓词 IN 用于判断属性值是否属于指定集合中的元素。如果属性值属于该集合，则返回“真”。

【查询 10】查询计算机科学系（CS）和计算机工程系（CE）的学生姓名和性别。

```
SELECT sname, sgender
FROM student
WHERE sdept IN ('CS', 'CE');
sname | sgender
-------+---------
 张三  | 男
 李四  | 女
 王五  | 女
(3 rows)
```

与 IN 相反的谓词是 NOT IN，它用于判断属性值是否不属于指定集合中的元素。如

果属性值不属于该集合，则返回“真”。

【查询 11】查询不是计算机科学系（CS）和计算机工程系（CE）的学生姓名、性别和所在系。

```
SELECT sname, sgender, sdept
FROM student
WHERE sdept NOT IN ('CS', 'CE');
sname | sgender | sdept
-------+---------+-------
 赵六  | 男      | IS
(1 row)
```

(4) 字符匹配

谓词 LIKE 用于进行字符串模式匹配。LIKE 的语法格式为：

<字符串> LIKE <模式> [ESCAPE <转义符>]

谓词 LIKE 判断属性值<字符串>与<模式>是否相匹配。实际上，<模式>定义了一个字符串集合。如果表示属性值的<字符串>包含在<模式>定义的这个字符串集合中，则 LIKE 谓词返回“真”。

<模式>的写法也是一个字符串，其中可以含有通配符_和%：

- _（下划线）代表任意单个字符。
- %（百分号）代表由 0 个或多个字符组成的任意字符串。

【查询 12】查询所有姓张的学生的学号、姓名和性别。

```
SELECT sno, sname, sgender
FROM student
WHERE sname LIKE '张%';
  sno  | sname | sgender
-------+-------+---------
 22001 | 张三  | 男
(1 row)
```

【查询 13】查询所有姓张的且姓名为两个字的学生的学号、姓名和性别。

```
SELECT sno, sname, sgender
FROM student
WHERE sname LIKE '张_';
  sno  | sname | sgender
-------+-------+---------
 22001 | 张三  | 男
(1 row)
```

注意，这里数据库采用 Unicode 字符编码，故'张_'中的下划线匹配一个汉字。openGauss 采用的字符编码可以查看 server_encoding 系统参数，命令如下：

```
dbsc=# SHOW server_encoding;
 server_encoding
-----------------
 UTF8
(1 row)
```

谓词 NOT LIKE 与 LIKE 相反，用于查找属性值与<模式>不匹配的元组。LIKE 返回“真”时，NOT LIKE 返回“假”，反之亦然。

【查询 14】查询所有不姓张的学生的学号、姓名和性别。

```
SELECT sno, sname, sgender
FROM student
WHERE sname NOT LIKE '张%';
  sno  | sname | sgender
-------+-------+---------
 22002 | 李四  | 女
 22003 | 王五  | 女
 22004 | 赵六  | 男
(3 rows)
```

如果要匹配的字符串中本身就含有_或%字符，要对通配符_或%进行转义，将其还原为字面含义，即下划线或百分号，而不再是通配符。默认的转义符为反斜线“\”。

【查询 15】查询课程名为 DB_Course 的课程的课号和学分。

```
SELECT cno, ccredit
FROM course
WHERE cname LIKE 'DB\_Course';
cno | ccredit
-----+---------
(0 rows)
```

可以使用 ESCAPE 关键字指定某字符为转义符。

【查询 16】查询课程名以 DB_开头且倒数第 2 个字符为 s 的课程的课号和学分。

```
SELECT cno, ccredit
FROM course
WHERE cname LIKE 'DB^_%s_' ESCAPE '^';
cno | ccredit
-----+---------
(0 rows)
```

可以使用正则表达式完成更复杂的字符匹配需求。正则表达式定义一个字符串集合。如果一个字符串是正则表达式所描述的集合的元素时，就说这个串匹配该正则表达式。openGauss 支持符合 POSIX 语法的正则表达式。表 2.8 给出了 openGauss 支持的正则表达式匹配运算符。

表 2.8 openGauss 支持的正则表达式匹配运算符

运算符	描述	例子
~	匹配正则表达式，大小写敏感	'openGauss' ~ '.*openGauss.*'
~*	匹配正则表达式，大小写不敏感	'openGauss' ~* '.*opengauss.*'
!~	不匹配正则表达式，大小写敏感	'openGauss' !~ '.*opengauss.*'
!~*	不匹配正则表达式，大小写不敏感	'openGauss' !~* '.*openeuler.*'

【查询 17】查询所在系的名称以 C 开头的学生的学号、姓名和系名。

```
SELECT sno, sname, sdept
FROM student
WHERE sdept ~ '^C.*';
  sno  | sname | sdept
-------+-------+-------
```

```
 22001 | 张三  | CS
 22002 | 李四  | CS
 22003 | 王五  | CE
(3 rows)
```

(5) 判断空值

谓词 IS NULL 用于判断属性值是否为空，若属性值为空，则返回“真”。

【查询 18】查询成绩为空的学生的学号和所选课程的课号。

```
SELECT sno, cno
FROM sc
WHERE score IS NULL;
sno | cno
-----+-----
(0 rows)
```

注意，不要写作 score = NULL。

相反地，谓词 IS NOT NULL 用于判断属性值是否为非空，若属性值为非空，则返回“真”。

【查询 19】查询成绩不为空的学生的学号和所选课程的课号。

```
SELECT sno, cno
FROM sc
WHERE score IS NOT NULL;
  sno  | cno
-------+-----
 22001 |   1
 22001 |   2
 22001 |   3
 22002 |   2
 22002 |   3
(5 rows)
```

(6) 逻辑运算

逻辑运算符 AND 和 OR 可用来联结多个简单查询条件，从而构成复杂查询条件。

【查询 20】查询计算机科学系（CS）的男生信息。

```
SELECT *
FROM student
WHERE sdept = 'CS' AND sgender = '男';
  sno  | sname | sgender |       sbirth        | sdept
-------+-------+---------+---------------------+-------
 22001 | 张三  | 男      | 2002-05-19 00:00:00 | CS
(1 row)
```

【查询 21】查询计算机科学系（CS）、计算机工程系（CE）和信息科学系（IS）的学生姓名和性别。

```
SELECT sname, sgender
FROM student
WHERE sdept = 'CS' OR sdept = 'CE' OR sdept = 'IS';
sname | sgender
```

```
-------+---------
 张三  | 男
 李四  | 女
 王五  | 女
 赵六  | 男
(4 rows)
```

3. 排序。

ORDER BY 子句对查询结果按照指定列的升序或降序排列。如果没有指定 ORDER BY 子句，则结果表行的顺序可能是随意的。如要确保查询结果的输出顺序，需要使用 ORDER BY。

【查询 22】查询选修了 2 号课程的学生学号及其成绩，按分数降序排列。

```
SELECT sno, score
FROM sc
WHERE cno = 2
ORDER BY score DESC;
  sno  | score
-------+-------
 22002 | 90
 22001 | 85
(2 rows)
```

ASC 关键字指定排序方向为升序，DESC 关键字指定排序方向为降序。如果不指定排序方向，则默认为升序。

【查询 23】查询全体学生的信息，查询结果按照所在系的名称升序排列，同一个系的学生按照姓名的降序排列。

```
SELECT *
FROM student
ORDER BY sdept ASC, sname COLLATE "zh_CN.utf8" DESC;
  sno  | sname | sgender |       sbirth        | sdept
-------+-------+---------+---------------------+-------
 22003 | 王五  | 女      | 2001-12-08 00:00:00 | CE
 22001 | 张三  | 男      | 2002-05-19 00:00:00 | CS
 22002 | 李四  | 女      | 2002-01-09 00:00:00 | CS
 22004 | 赵六  | 男      | 2001-08-30 00:00:00 | IS
(4 rows)
```

升序将较小的值排在前面，降序将较大的值排在前面。字符串的升序就是字符串的字典序。因此，首先根据 sdept 列的升序排列，得到'CE' < 'CS' < 'IS'；对于 sdept 列值相同的行（即'CS'），再根据 sname 列的降序排列，得到'张三' > '李四'。注意，要使用 COLLATE "zh_CN.utf8"指明按照中文 UTF-8 字符集进行排序，才能保证按照中文拼音排序。

4. 聚合函数。

聚合函数的作用是由一组输入值计算出一个输出值。一组输入值一般是表在某列上的投影所产生的若干行，输出值只有一个。“聚合”的含义就是将一组值聚合在一起求出一个值。表 2.9 给出了 SQL 中常用的聚合函数。

表 2.9 常用的 SQL 聚合函数

聚合函数	描述
COUNT(*)	计算元组的个数
COUNT(<列名>)	计算一列中值的个数
SUM(<列名>)	计算一列值的总和（该列必须是数值型）
AVG(<列名>)	计算一列值的平均值（该列必须是数值型）
MAX(<列名>)	求一列值中的最大值。
MIN(<列名>)	求一列值中的最小值。

【查询 24】查询学生的总人数。

```
SELECT COUNT(*)
FROM student;
count
-------
     4
(1 row)
```

一个学生可以选修多门课程，sc 表中可能存在多条选课记录。要查询选修了课程的学生人数，同一学生的选课记录不能重复计算。要在聚合函数中使用 DISTINCT 关键字，在计算聚合函数之前消除指定列中的重复值。

【查询 25】查询选修了课程的学生人数。

```
SELECT COUNT(DISTINCT sno)
FROM sc;
  count
-------
     2
(1 row)
```

【查询 26】计算 3 号课程的平均成绩。

```
SELECT ROUND(AVG(score), 2) AS avg
FROM sc
WHERE cno = '3';
  avg
-------
 84.00
(1 row)
```

ROUND 函数用于将平均成绩舍入为保留两位小数。

【查询 27】计算 3 号课程的最高分数。

```
SELECT MAX(score)
FROM sc
WHERE cno = '3';
  max
-------
 88
(1 row)
```

【查询 28】计算学生 22001 所选课程的总成绩。

```
SELECT SUM(score)
```

```
FROM sc
WHERE sno = '22001';
 sum
-----
 269
(1 row)
```

5. 分组。

GROUP BY 子句可将查询结果按某些指定列的值进行分组，值相等的行分为一组。分组的目的往往是为了细化聚合函数的作用对象。如果未分组，聚合函数将作用于整个查询结果。分组后聚合函数将作用于每一个组，即每一组都有一个聚合函数值。

【查询 29】求各个课程号及相应的选课人数。

```
SELECT cno, COUNT(sno)
FROM sc
GROUP BY cno;
 cno | count
-----+-------
   1 |     1
   3 |     2
   2 |     2
(3 rows)
```

该查询对结果按 cno 分组，具有相同 cno 值的行分为一组，对每一组使用聚合函数 COUNT 进行计数。

若只对分组后的某些组感兴趣，可使用 HAVING 关键字指定筛选条件，筛选出满足指定条件的组。

【查询 30】查询至少选修了 3 门课程的学生学号。

```
SELECT sno
FROM sc
GROUP BY sno
HAVING COUNT(*) >= 3;
  sno
-------
 22001
(1 row)
```

该查询先用 GROUP BY 按 sno 进行分组，再用聚合函数 COUNT(*)对每一组计数。用 HAVING 指定要选出的组须满足的条件，即 COUNT(*) >= 3，表示此学生选修的课程大于等于 3 门。

2.4.3 查询数据——连接查询

之前的查询只涉及一张数据表。当需要获取的数据来自两个以上表时，应执行连接查询。SQL 提供的连接查询分为内连接（inner join）和外连接（outer join）两大类型。

1. 内连接。

将两个表连接在一起的条件称为连接谓词（join predicate）或连接条件。内连接只

返回两个表中与连接谓词匹配的行，不匹配的行不会被输出。
内连接包括：等值连接（equi-join）和交叉连接（cross join）。
(1) 等值连接
等值连接是指连接谓词中只含有=运算符的内连接，其语法格式为：

```
SELECT <列名>, …
FROM <左表> INNER JOIN <右表>
    ON <左表>.<主键> = <右表>.<外键>
```

等值连接是最常用的一种连接查询类型。在 FROM 子句中指定两个待连接的表和连接谓词。INNER JOIN 关键字表示进行内连接，关键字左侧的表称为左表，右侧的表称为右表。ON 关键字后面是连接谓词。通常情况下，等值连接的谓词是一个表的主键等于另一个表的外键。正是通过数据库中定义的主外键关系，表与表才能按数据查询需求连接在一起。连接谓词中出现的列称为连接字段。
【查询 31】查询每个学生及其选课情况。
学生信息在 student 表中，选课情况在 sc 表中，本查询需涉及两个表。两个表之间的联系是通过 student 表的主键 sno 和 sc 表的外键 sno 的主外键关联实现的。

```
SELECT *
FROM student INNER JOIN sc
    ON student.sno = sc.sno;
  sno  | sname | sgender |       sbirth        | sdept |  sno  | cno | score
-------+-------+---------+---------------------+-------+-------+-----+-------
 22001 | 张三  | 男      | 2002-05-19 00:00:00 | CS    | 22001 |   1 | 96
 22001 | 张三  | 男      | 2002-05-19 00:00:00 | CS    | 22001 |   2 | 85
 22001 | 张三  | 男      | 2002-05-19 00:00:00 | CS    | 22001 |   3 | 88
 22002 | 李四  | 女      | 2002-01-09 00:00:00 | CS    | 22002 |   2 | 90
 22002 | 李四  | 女      | 2002-01-09 00:00:00 | CS    | 22002 |   3 | 80
(5 rows)
```

内连接的连接谓词也可在 WHERE 子句中指定。这种形式的内连接是旧式语法。与上述查询等价的旧式语法内连接为：

```
SELECT *
FROM student, sc
WHERE student.sno = sc.sno;
```

建议使用 JOIN 关键字指定连接表和连接谓词，可以清晰地表示出连接操作。
上述结果表中，存在两个 sno 列，它们正是连接字段，一个是 student 表的主键 sno，一个是 sc 表的外键 sno。通过直接指定输出列名，可以去掉结果表中的重复列。在结果表中只出现一次连接字段的等值连接即为自然连接（natural join）。
【查询 32】使用自然连接查询每个学生及其选课情况。

```
SELECT student.sno, sname, sgender, sbirth, sdept, cno, score
FROM student INNER JOIN sc
    ON student.sno = sc.sno;
  sno  | sname | sgender |       sbirth        | sdept | cno | score
-------+-------+---------+---------------------+-------+-----+-------
 22001 | 张三  | 男      | 2002-05-19 00:00:00 | CS    |   1 | 96
 22001 | 张三  | 男      | 2002-05-19 00:00:00 | CS    |   2 | 85
```

```
 22001 | 张三  | 男      | 2002-05-19 00:00:00 | CS    |   3 | 88
 22002 | 李四  | 女      | 2002-01-09 00:00:00 | CS    |   2 | 90
 22002 | 李四  | 女      | 2002-01-09 00:00:00 | CS    |   3 | 80
(5 rows)
```

(2) 交叉连接

交叉连接操作返回两个表的笛卡尔积。可以将交叉连接理解为连接谓词总为真的内连接。这样，左表的每一行都能与右表的全部行连接起来，形成的结果表的行数是左右两个表行数的乘积。交叉连接使用 CROSS JOIN 关键字实现。

【查询 33】对 student 表和 sc 表进行交叉连接操作。

```
SELECT *
FROM student CROSS JOIN sc;
  sno  | sname | sgender |       sbirth        | sdept |  sno  | cno | score
-------+-------+---------+---------------------+-------+-------+-----+-------
 22001 | 张三  | 男      | 2002-05-19 00:00:00 | CS    | 22001 |   1 | 96
 22002 | 李四  | 女      | 2002-01-09 00:00:00 | CS    | 22001 |   1 | 96
 22003 | 王五  | 女      | 2001-12-08 00:00:00 | CE    | 22001 |   1 | 96
 22004 | 赵六  | 男      | 2001-08-30 00:00:00 | IS    | 22001 |   1 | 96
 22001 | 张三  | 男      | 2002-05-19 00:00:00 | CS    | 22001 |   2 | 85
 22002 | 李四  | 女      | 2002-01-09 00:00:00 | CS    | 22001 |   2 | 85
 22003 | 王五  | 女      | 2001-12-08 00:00:00 | CE    | 22001 |   2 | 85
 22004 | 赵六  | 男      | 2001-08-30 00:00:00 | IS    | 22001 |   2 | 85
 22001 | 张三  | 男      | 2002-05-19 00:00:00 | CS    | 22001 |   3 | 88
 22002 | 李四  | 女      | 2002-01-09 00:00:00 | CS    | 22001 |   3 | 88
 22003 | 王五  | 女      | 2001-12-08 00:00:00 | CE    | 22001 |   3 | 88
 22004 | 赵六  | 男      | 2001-08-30 00:00:00 | IS    | 22001 |   3 | 88
 22001 | 张三  | 男      | 2002-05-19 00:00:00 | CS    | 22002 |   2 | 90
 22002 | 李四  | 女      | 2002-01-09 00:00:00 | CS    | 22002 |   2 | 90
 22003 | 王五  | 女      | 2001-12-08 00:00:00 | CE    | 22002 |   2 | 90
 22004 | 赵六  | 男      | 2001-08-30 00:00:00 | IS    | 22002 |   2 | 90
 22001 | 张三  | 男      | 2002-05-19 00:00:00 | CS    | 22002 |   3 | 80
 22002 | 李四  | 女      | 2002-01-09 00:00:00 | CS    | 22002 |   3 | 80
 22003 | 王五  | 女      | 2001-12-08 00:00:00 | CE    | 22002 |   3 | 80
 22004 | 赵六  | 男      | 2001-08-30 00:00:00 | IS    | 22002 |   3 | 80
(20 rows)
```

如使用旧式连接语法，交叉连接就是没有 WHERE 子句的内连接。换言之，内连接的结果集可以看作是在交叉连接的结果上用 WHERE 子句施加过滤条件而构成的。与上面查询等价的旧式语法交叉连接为：

```
SELECT *
FROM student, sc;
```

2. 外连接。

如果希望连接操作将左表或右表中不匹配的行也输出，需要使用外连接。外连接分为左外连接（left outer join）、右外连接（right outer join）和完全外连接（full outer join）。外连接使用 OUTER JOIN 关键字。

(1) 左外连接

左外连接的结果会返回左表中全部行（即使在右表中没有找到匹配的行）。如果左表中某行在右表中没有匹配的行，则在结果表中对应的行上，来自右表的列填充空值（NULL）。左外连接使用 LEFT OUTER JOIN 关键字。

【查询 34】查询每个学生及其选修课程的情况，没有选课的学生也要列出。

```
\pset null 'NULL'


SELECT *
FROM student LEFT OUTER JOIN sc
    ON student.sno = sc.sno;
  sno  | sname | sgender |       sbirth        | sdept |  sno  | cno  | score
-------+-------+---------+---------------------+-------+-------+------+-------
 22001 | 张三  | 男      | 2002-05-19 00:00:00 | CS    | 22001 |    1 | 96
 22001 | 张三  | 男      | 2002-05-19 00:00:00 | CS    | 22001 |    2 | 85
 22001 | 张三  | 男      | 2002-05-19 00:00:00 | CS    | 22001 |    3 | 88
 22002 | 李四  | 女      | 2002-01-09 00:00:00 | CS    | 22002 |    2 | 90
 22002 | 李四  | 女      | 2002-01-09 00:00:00 | CS    | 22002 |    3 | 80
 22003 | 王五  | 女      | 2001-12-08 00:00:00 | CE    | NULL  | NULL | NULL
 22004 | 赵六  | 男      | 2001-08-30 00:00:00 | IS    | NULL  | NULL | NULL
(7 rows)
```

gsql 命令\pset null 'NULL'是设置 NULL 的输出显示，gsql 默认以空串的形式表示 NULL。

(2) 右外连接

与左外连接相对应，右外连接的结果会返回右表中全部行，即使在左表中没有找到匹配的行。如果右表中的某行在左表中没有匹配的行，那么在连接结果表中对应的行上，来自左表的属性为空值（NULL）。右外连接使用 RIGHT OUTER JOIN 关键字。

【查询 35】查询选课情况及相应每门课程信息，没有被选的课程也要列出。

```
SELECT *
FROM sc RIGHT OUTER JOIN course
    ON sc.cno = course.cno;
  sno  | cno  | score | cno |   cname    | ccredit
-------+------+-------+-----+------------+---------
 22001 |    1 | 96    |   1 | 高等数学   |       4
 22001 |    2 | 85    |   2 | 数据库系统 |       5
 22001 |    3 | 88    |   3 | 操作系统   |       3
 22002 |    2 | 90    |   2 | 数据库系统 |       5
 22002 |    3 | 80    |   3 | 操作系统   |       3
 NULL  | NULL | NULL  |   4 | 信息系统   |       4
(6 rows)
```

(3) 完全外连接

完全外连接将左外连接和右外连接的结果组合在一起，形成的结果表包含左表和右表中全部的行，没有匹配上的列值使用 NULL 填充。完全外连接使用 FULL OUTER JOIN 关键字。

为了演练完全外连接，在“学生-课程”数据库中创建系别表 department 并插入数据。

```
CREATE TABLE department
```

```
(
    id      CHAR(2),
    name    VARCHAR(20)
);
INSERT INTO department VALUES('CS', '计算机科学');
INSERT INTO department VALUES('CE', '计算机工程');
INSERT INTO department VALUES('IS', '信息科学');
INSERT INTO department VALUES('SE', '软件工程');
```

同时，在 student 表中增加一条学生记录，如下：

```
INSERT INTO Student VALUES ('22005', '孙七', '男', '2001-11-01', NULL);
```

【查询 36】查询每个系及其中学生的情况，要求同时列出尚无学生的系和尚未分配系的学生。

```
SELECT *
FROM department FULL OUTER JOIN student
    ON department.id = student.sdept;
  id  |    name    |  sno  | sname | sgender |       sbirth        | sdept
------+------------+-------+-------+---------+---------------------+-------
 CS   | 计算机科学 | 22002 | 李四  | 女      | 2002-01-09 00:00:00 | CS
 CS   | 计算机科学 | 22001 | 张三  | 男      | 2002-05-19 00:00:00 | CS
 CE   | 计算机工程 | 22003 | 王五  | 女      | 2001-12-08 00:00:00 | CE
 IS   | 信息科学   | 22004 | 赵六  | 男      | 2001-08-30 00:00:00 | IS
 SE   | 软件工程   | NULL  | NULL  | NULL    | NULL                | NULL
 NULL | NULL       | 22005 | 孙七  | 男      | 2001-11-01 00:00:00 | NULL
(6 rows)
```

可见，软件工程系目前没有学生，同时，学生孙七也还没有被分配系。

3. 自连接。

自连接（self-join）是一个表与其自身进行连接操作。自连接并不是一种新的连接类型，是一种特殊的内连接。

为了演练自连接，创建新表 student_p，其在 student 表基础上增加了表示省份的 province 列。

```
CREATE TABLE student_p
(
    sno         CHAR(5),
    sname       VARCHAR(10) NOT NULL,
    sgender     CHAR(4)     NOT NULL,
    sbirth      DATE        NOT NULL,
    sdept       CHAR(2),
    province    VARCHAR(10),
    CONSTRAINT pk_student_p PRIMARY KEY(sno)
);

INSERT INTO student_p VALUES ('22001', '张三', '男', '2002-05-19', 'CS', '北京');
INSERT INTO student_p VALUES ('22002', '李四', '女', '2002-01-09', 'CS', '天津');
INSERT INTO student_p VALUES ('22003', '王五', '女', '2001-12-08', 'CE', '河北');
```

```
INSERT INTO student_p VALUES ('22004', '赵六', '男', '2001-08-30', 'IS', '北京');
```

【查询 37】列出来自相同省份的学生学号、姓名和省份。

将 student_p 表与其自身连接，需要分出一个表的两个副本，以给表起别名的方式实现。

```
SELECT s1.sno, s1.sname, s2.sno, s2.sname, s1.province
FROM student_p s1 INNER JOIN student_p s2
    ON s1.province = s2.province
WHERE s1.sno <> s2.sno
ORDER BY s1.sno, s2.sno;
  sno  | sname |  sno  | sname | province
-------+-------+-------+-------+----------
 22001 | 张三  | 22004 | 赵六  | 北京
 22004 | 赵六  | 22001 | 张三  | 北京
(2 rows)
```

WHERE 条件 s1.sno <> s2.sno 表示过滤掉一个学生与自己匹配的行，但观察结果表发现，输出的两行只是左右表顺序颠倒，信息内容一样。将 WHERE 条件改为 s1.sno < s2.sno 可以解决这一问题。

4. 多表连接。

多表连接即两张以上的表进行连接操作。多表连接需要使用多个 JOIN 关键字。

【查询 38】查询每个学生的学号、姓名、选修的课程名及成绩。

学生的学号和姓名在 student 表中，课程名在 course 表中，成绩在 sc 表中。本查询涉及 3 张表，应使用多表连接完成。

```
SELECT s.sno, s.sname, c.cname, sc.score
FROM student s INNER JOIN sc ON s.sno = sc.sno
               INNER JOIN course c ON sc.cno = c.cno;
  sno  | sname |   cname    | score
-------+-------+------------+-------
 22001 | 张三  | 高等数学   | 96
 22001 | 张三  | 数据库系统 | 85
 22001 | 张三  | 操作系统   | 88
 22002 | 李四  | 数据库系统 | 90
 22002 | 李四  | 操作系统   | 80
(5 rows)
```

由于内连接运算满足交换律和结合律，多张表的连接顺序不会改变结果表。所以 DBMS 不一定按照表的先后顺序依次执行每个连接，而是根据执行代价选择最优的连接顺序。

2.4.4 查询数据——子查询

查询块是一个 SELECT-FROM-WHERE 语句结构。将一个查询块嵌套在另一个查询块中所构成的查询称为嵌套子查询。例如，

```
SELECT sname            /* 外层查询，父查询 */
FROM student
```

```
WHERE sno IN (
    SELECT sno          /* 内层查询，子查询 */
    FROM sc
    WHERE cno = 2
    )
```

括号中的内层查询块嵌套在外层查询块的 WHERE 条件中。外层查询块称为**外层查询**或**父查询**，内层查询块称为**内层查询**或**子查询**。可用子查询结构将多个简单查询块构造为复杂查询，从而分步求解一个复杂的查询问题。

1. IN 谓词子查询。

   子查询的结果往往是一个集合，这时，外层查询可用谓词 IN 连接内层查询。

   【查询 39】查询与“李四”在同一个系学习的学生。

   按照子查询思路编写此查询的步骤是，先来完成简单子查询，再构造嵌套查询。

   ① 确定“李四”所在的系的名称。

```
SELECT sdept
FROM student
WHERE sname = '李四';
sdept
-------
 CS
(1 row)
```

   ② 查找所有在 CS 系学习的学生。

```
SELECT sno, sname, sdept
FROM student
WHERE sdept = 'CS';
  sno  | sname | sdept
-------+-------+-------
 22001 | 张三  | CS
 22002 | 李四  | CS
(2 rows)
```

   使用 IN 谓词，将第①步查询块嵌入到第②步查询块的 WHERE 条件中，构造如下嵌套查询：

```
SELECT sno, sname, sdept
FROM student
WHERE sdept IN (
    SELECT sdept
    FROM student
    WHERE sname = '李四'
);
  sno  | sname | sdept
-------+-------+-------
 22001 | 张三  | CS
 22002 | 李四  | CS
(2 rows)
```

   本查询是**不相关子查询，其**子查询的查询条件不依赖于父查询，因而可先执行子查

询，再执行父查询。在不相关子查询中，子查询的结果用于建立父查询的查找条件。

【查询 40】查询选修了“数据库系统”课程的学生学号和姓名。

本查询涉及学号、姓名和课程名，学号和姓名存放在是 student 表中，课程名存放在 course 表中。student 表与 course 表之间没有直接联系，需要通过 sc 表来建立二者之间的联系。因而本查询涉及 3 张表。使用子查询实现如下：

```
SELECT sno, sname        /* 3. 在 student 表中取出 sno 和 sname */
FROM student
WHERE sno IN (
    SELECT sno               /* 2. 在 sc 表中找出选修了 2 号课程的学生的学号 */
    FROM sc
    WHERE cno IN (
            SELECT cno            /* 1. 在 course 表中找出“数据库系统”的课程号为 2 */
            FROM course
            WHERE cname = '数据库系统'
    )
);
  sno  | sname
-------+-------
 22001 | 张三
 22002 | 李四
(2 rows)
```

本查询问题也可以用连接查询来实现。

```
SELECT s.sno, s.sname
FROM student s INNER JOIN sc ON s.sno = sc.sno
               INNER JOIN course c ON sc.cno = c.cno
WHERE c.cname = '数据库系统';
```

2. 比较运算符子查询。

父查询与子查询之间用比较运算符连接，构成带有比较运算符的子查询。

【查询 41】找出每个学生超过他选修课程平均成绩的课程号。

```
SELECT sno, cno
FROM sc x
WHERE score > (
     SELECT AVG(score)
     FROM sc y
     WHERE y.sno = x.sno
);
  sno  | cno
-------+-----
 22001 |   1
 22002 |   2
(2 rows)
```

本查询可以这样理解，父查询中 sc 表的别名为 x，可以认为 x 表示 sc 表中的一行，子查询计算一个学生所有选修课程的平均成绩，至于是哪个学生，就要看 x.sno 的值了，该值取决于父查询中当前 x 代表哪个元组。因此，子查询的查询条件依赖于

父查询，这类子查询称为**相关子查询**。在相关子查询中，子查询与父查询相关联，因而要多次求解子查询。

3. ANY 或 ALL 谓词子查询

如果子查询返回多个值，连接子查询的比较运算符前面要用 ANY 或 ALL 谓词修饰。ANY 表示“某个、其中之一”，ALL 表示“所有、全部”。表 2.10 列出了 ANY 和 ALL 与 6 种比较运算符搭配而构成的 12 个谓词。

表 2.10 ANY 和 ALL 谓词的语义

谓词	语义
>ANY	大于某个值
>ALL	大于所有值
<ANY	小于某个值
<ALL	小于所有值
>=ANY	大于等于某个值
>=ALL	大于等于所有值
<=ANY	小于等于某个值
<=ALL	小于等于所有值
=ANY	等于某个值
=ALL	等于所有值
<>ANY	不等于某个值
<>ALL	不等于所有值

【查询 42】查询其他系中比 IS 系某一学生出生日期晚的学生姓名、出生日期和所在系。

```
SELECT sname, sbirth, sdept
FROM student
WHERE sbirth > ANY (
     SELECT sbirth
     FROM student
     WHERE sdept = 'IS'
)
AND sdept <> 'IS';
sname |       sbirth        | sdept
-------+---------------------+-------
 张三  | 2002-05-19 00:00:00 | CS
 李四  | 2002-01-09 00:00:00 | CS
 王五  | 2001-12-08 00:00:00 | CE
(3 rows)
```

本查询首先执行子查询，找出 IS 系中所有学生的出生日期，构成一个集合；然后处理父查询，查找不在 IS 系且出生日期大于（即晚于）子查询结构集合中某一元素的学生。

本查询也可用聚合函数来实现。实际上，大于某个值等价于大于最小值。首先用子查询找出 IS 系学生的最早出生日期（即最小值），然后在父查询中查找所有非 IS 系且出生日期大于该最小值的学生。具体查询如下：

```
SELECT sname, sbirth, sdept
```

```
FROM student
WHERE sbirth > (
    SELECT MIN(sbirth)
    FROM student
    WHERE sdept = 'IS'
)
AND sdept <> 'IS';
```

【查询 43】查询其他系中比 IS 系所有学生出生日期都晚的学生姓名、出生日期和所在系。

```
SELECT sname, sbirth, sdept
FROM student
WHERE sbirth > ALL (
     SELECT sbirth
     FROM student
     WHERE sdept = 'IS'
)
AND sdept <> 'IS';
```

大于所有值等价于大于最大值，因此本查询也可以用聚合函数实现如下：

```
SELECT sname, sbirth, sdept
FROM student
WHERE sbirth > (
    SELECT MAX(sbirth)
    FROM student
    WHERE sdept = 'IS'
)
AND sdept <> 'IS';
```

谓词之间存在着若干等价关系，例如，=ANY 等价于 IN 谓词，<>ALL 等价于 NOT IN 谓词，>ANY 等价于>MIN()，>ALL 等价于>MAX()等，如表 2.11 所示。

表 2.11 ANY 和 ALL 谓词与聚合函数和 IN 谓词的等价关系

	=	<>	<	<=	>	>=
ANY	IN	--	<MAX	<=MAX	>MIN	>=MIN
ALL	--	NOT IN	<MIN	<=MIN	>MAX	>=MAX

4. EXISTS 谓词子查询

EXISTS 称为存在量词。EXISTS 引出的子查询相当于进行存在测试，即父查询测试子查询是否返回结果。子查询不产生任何数据，只返回逻辑真假值 TRUE 或 FALSE。如何子查询能够返回至少一行，则 EXISTS 谓词结果为“真”；否则，结果为“假”。

【查询 44】查询所有选修了 1 号课程的学生姓名。

思路是在 student 表中依次取出每个元组的 sno 值，用此值去检查 sc 表。若 sc 表中存在这样的元组，其 sno 值等于此 student.sno 值并且其 cno=2，则取此 student.sname 放入结果中。实现此思路的查询如下：

```
SELECT sname
FROM student
WHERE EXISTS (
```

```
    SELECT *
    FROM sc
    WHERE sc.sno = student.sno AND cno = 2
);
sname
-------
 张三
 李四
(2 rows)
```

EXISTS 谓词只关心子查询的结果是否存在，不关心结果的具体内容，因此子查询 SELECT 后面通常为*。这里子查询的查询条件 sc.sno = student.sno 依赖于父查询中的 student.sno 值，因此也是相关子查询。

与 EXISTS 谓词相反，NOT EXISTS 谓词表示不存在。若 NOT EXISTS 引导的子查询没有结果，则父查询的 WHERE 子句返回真值，否则返回假值。

【查询 45】查询没有选修 1 号课程的学生姓名。

```
SELECT sname
FROM student
WHERE NOT EXISTS (
    SELECT *
    FROM sc
    WHERE sc.sno = student.sno AND cno = 2
);
sname
-------
 王五
 赵六
(2 rows)
```

SQL 中没有全称量词，使用两个 NOT EXISTS 可以表示全称量词。

【查询 46】查询选修了全部课程的学生的学号和姓名。

用两个否定的存在量词 NOT EXISTS 构造嵌套的子查询，表示等价的存在量词，即将查询问题转化为：查询这样的学生，没有一门课程是他不选的。

```
SELECT s.sno, s.sname
FROM student AS s
WHERE NOT EXISTS (       /* 没有一门课程 */
    SELECT *
    FROM course AS c
    WHERE NOT EXISTS (   /* 是该学生不选的 */
        SELECT *
        FROM sc
        WHERE sc.sno = s.sno AND sc.cno = c.cno
    )
);
```

该查询没有返回结果，说明目前还不存在选修了全部课程的学生。

这时，在 sc 表中添加一条记录：

```
INSERT INTO sc VALUES ('22001', 4, NULL);
```

即 22001 这个学生选修了 4 号课。再次执行上面查询，得到结果：

```
  sno   | sname
-------+-------
 22001 | 张三
(1 row)
```

即学生张三选修了全部课程。

2.4.5 查询数据——集合查询

SELECT 语句的查询结果是元组的集合，两个查询结果可以进行集合操作。集合操作包括集合并操作 UNION、集合交操作 INTERSECT 和集合差操作 EXCEPT。语法格式为：

<查询 1> UNION <查询 2>

<查询 1> INTERSECT <查询 2>

<查询 1> EXCEPT <查询 2>

参加集合操作的<查询 1>和<查询 2>的结果列数和对应列的数据类型必须兼容。集合操作可以扩展到多个查询之间，例如，

<查询 1> UNION <查询 2> UNION <查询 3>

【查询 47】查询 CS 系的学生及所有男学生。

本查询是求 CS 系的学生与所有男学生的并集。UNION 执行查询结果的集合并操作，会自动去掉重复的元组。

```
SELECT *
FROM student
WHERE sdept = 'CS'
UNION
SELECT *
FROM student
WHERE sgender = '男';
  sno   | sname | sgender |       sbirth        | sdept
-------+-------+---------+---------------------+-------
 22004 | 赵六  | 男      | 2001-08-30 00:00:00 | IS
 22001 | 张三  | 男      | 2002-05-19 00:00:00 | CS
 22002 | 李四  | 女      | 2002-01-09 00:00:00 | CS
(3 rows)
```

【查询 48】查询 CS 系的男学生。

该查询即查找 CS 系学生与男学生的交集。INTERSECT 执行查询结果的集合交操作。

```
SELECT *
FROM student
WHERE sdept = 'CS'
INTERSECT
SELECT *
FROM student
```

```
WHERE sgender = '男';
  sno  | sname | sgender |       sbirth        | sdept
-------+-------+---------+---------------------+-------
 22001 | 张三  | 男      | 2002-05-19 00:00:00 | CS
(1 row)
```

该查询使用复合条件实现为:

```
SELECT *
FROM student
WHERE sdept = 'CS' AND sgender = '男';
```

【查询 49】查询不在 CS 系的女学生。

该查询是查找女学生与 CS 系学生的差集。EXCEPT 执行查询结果的集合交操作。

```
SELECT *
FROM student
WHERE sgender = '女'
EXCEPT
SELECT *
FROM student
WHERE sdept = 'CS';
  sno  | sname | sgender |       sbirth        | sdept
-------+-------+---------+---------------------+-------
 22003 | 王五  | 女      | 2001-12-08 00:00:00 | CE
(1 row)
```

2.4.6 更新数据

1. 插入数据

使用 INSERT 语句向指定表中插入一行数据。

【查询 50】将一条新学生记录

（学号：22005，姓名：孙七，性别：男，出生日期：2001-11-01， 系别：CE）插入到 student 表中。

```
INSERT INTO student
VALUES ('22005', '孙七', '男', '2001-11-01', 'CE');
```

INSERT INTO 后面指定了表名 student，但省略了列名，表示为表中所有列赋值，这种情况下插入行数据必须在每个具有 NOT NULL 约束的列上都有值，且顺序要与表中列的顺序相一致。如果指定了列名，则值的顺序必须与对应的列一致。例如，

```
INSERT INTO student (sno, sname, sgender, sdept, sbirth)
VALUES ('22005', '孙七', '男', 'CE', '2001-11-01');
```

这里 sdept 列在前 sbirth 列在后，不同于表定义中列的顺序。

【查询 51】插入一条选课记录('22005', 4)。

```
INSERT INTO sc (sno, cno)
VALUES ('22005', 4);
```

因为没有提供 score 列的值，数据库将在新插入行的 score 列上自动赋空值。或者等价地写作:

```
INSERT INTO sc
```

```
VALUES ('22005', 4, NULL);
```

因为没有指出 sc 表的列名，则在 score 列上要明确给出空值。

INSERT 语句中嵌套 SELECT 子查询，可将一个表中满足条件的行批量插入另一表中。

【查询 52】将 CS 系学生的学号和姓名存放到一张单独的表中。

首先建立一个新表 student_cs：

```
CREATE TABLE student_cs
(
    sno       CHAR(5),
    sname   VARCHAR(10)
);
```

然后查询 CS 系学生的学号和姓名，将结果插入到 student_cs 表中。

```
INSERT INTO student_cs(sno, sname)
    SELECT sno, sname
    FROM student
    WHERE sdept = 'CS';
```

查看 student_cs 表中数据：

```
SELECT * FROM student_cs;
  sno  | sname
-------+-------
 22001 | 张三
 22002 | 李四
(2 rows)
```

2. 修改数据

使用 UPDATE 语句进行数据修改操作。UPDATE 语句指明修改数据所需的三项信息。UPDATE 后面是表名，SET 子句指定要修改的列和修改后的新数据值，WHERE 子句指定修改行所满足的条件。如果没有 WHERE 子句，则表示要修改表中的所有行。

【查询 53】将学生 22005 的系别改为 IS。

```
UPDATE student
SET sdept = 'IS'
WHERE sno = '22005';
```

【查询 54】将所有学生所选课程成绩增加 2 分。

```
UPDATE sc
SET score = score + 1;
```

在 UPDATE 语句的 WHERE 条件中可以嵌套子查询，用以构造修改数据的条件。

【查询 55】将 CS 系全体学生的成绩设置为零。

```
UPDATE sc
SET score = 0
WHERE 'CS' = (
    SELECT sdept
    FROM student
    WHERE student.sno = sc.sno
);
```

3. 删除数据

使用 DELETE 语句进行数据删除操作。DELETE 语句从指定表中删除满足 WHERE 子

句条件的行。如果没有 WHERE 子句，表示删除表中全部行。

【查询 56】删除学号为 22005 的学生记录。

```
DELETE FROM student
WHERE sno = '22005';
ERROR:  update or delete on table "student" violates foreign key constraint "fk_student" on table "sc"
DETAIL:  Key (sno)=(22005) is still referenced from table "sc".
```

出现报错信息，原因是删除 student 表中的行违反 sc 表上的外键约束。故而要先删除 sc 表中 22005 学生的选课记录，执行如下语句：

```
DELETE FROM sc
WHERE sno = '22005';
```

然后，再次执行前面的删除 22005 学生记录的操作，返回操作成功。

【查询 57】删除所有的学生选课记录。

该操作就是要删除 sc 表的所有行，即清空 sc 表，可使用没有 WHERE 条件的 DELETE 语句。

```
DELETE FROM sc;
```

DELETE 语句中子也可以嵌套查询，用以构造删除数据的条件。

【查询 58】删除 CE 系所有学生的选课记录。

```
DELETE FROM sc
WHERE 'CE' = (
    SELECT sdept
    FROM student
    WHERE student.sno = sc.sno
);
```

2.4.7 修改表结构

有时需要对表结构本身进行修改，而不是修改表中的数据。一种办法是将整张表删除后再重新创建。如果表中已有大量数据，或者该表被其他表引用（例如外键约束），则删除表重建将带来很多不便。SQL 中的 ALTER TABLE 语句可以应对这种情况，仅对表结构进行修改，而保持表中数据不变。

1. 添加列

   【查询 59】在 course 表中添加 description 列，用于保存课程描述。

   ```
   ALTER TABLE course ADD COLUMN description TEXT;
   ```

   可以看到语句中指定了列类型 TEXT。实际上，在 CREATE TABLE 语句中可以进行的列定义，在 ALTER TABLE 语句中都可以指定。

2. 删除列

   【查询 60】将 description 列从 course 表中删除。

   ```
   ALTER TABLE course DROP COLUMN description;
   ```

   删除列时，该列中的数据会一并删除掉。但如果有其他表通过外键约束依赖该列，为了维护参照完整性，数据库将进行提示，而非直接删除该列。可以添加 CASCADE 关键字，在删除列时，一并删除所有依赖该列的数据库对象。

   ```
   ALTER TABLE course DROP COLUMN description CASCADE;
   ```

3. 添加约束

【查询 61】在 sc 表中添加约束，确保 score 大于等于 0 且小于等于 100。

```
ALTER TABLE sc ADD CHECK (score >= 0 AND score <= 100);
```

【查询 62】在 sc 表中添加约束，确保 score 列不为空值。

```
ALTER TABLE sc ALTER COLUMN score SET NOT NULL;
```

注意：添加非空约束的语法采用 ALTER COLUMN 和 SET NOT NULL。

数据库会检查表中已有数据是否满足准备添加的约束，如果不满足，则该约束会添加失败。

4. 删除约束

删除一个约束需要知道该约束的名称。

【查询 63】在 sc 表中删除外键约束 fk_student。

```
ALTER TABLE sc DROP CONSTRAINT fk_student;
```

同样，删除非空约束时，需要采用单独的语法。

【查询 64】在 sc 表中删除 score 列上的非空约束。

```
ALTER TABLE sc ALTER COLUMN score DROP NOT NULL;
```

5. 设置默认值

使用 ALTER TABLE 语句可以为一列设置默认值。

【查询 65】在 sc 表中，为 score 列设置默认值为 0。

```
ALTER TABLE sc ALTER COLUMN score SET DEFAULT 0;
```

注意，设置默认值不会改变表中该列的已有数值，其只会影响新插入行在该列上的默认值。

【查询 66】在 sc 表中，删除 score 列上设置的默认值。

```
ALTER TABLE sc ALTER COLUMN score DROP DEFAULT;
```

6. 修改列数据类型

【查询 67】在 sc 表中，将 score 列的数据类型改为 NUMERIC(5, 2)。

```
ALTER TABLE sc ALTER COLUMN score TYPE NUMERIC(5, 2);
```

注意，只有当该列的已有数据值能够通过默认类型转换转为指定的新数据类型时，修改列数据类型语句才能执行成功。

7. 重命名列

【查询 68】在 sc 表中，将 score 列名称改为 grade。

```
ALTER TABLE sc RENAME COLUMN score TO grade;
```

8. 重命名表

【查询 69】将 sc 表的名称改为 student_course。

```
ALTER TABLE sc RENAME TO student_course;
```

2.5 实验结果

【请按照要求完成实验操作】

1. 完成实验步骤 2.4.1 节。

2. 完成实验步骤 2.4.2 节。

3. 完成实验步骤 2.4.3 节。

4. 完成实验步骤 2.4.4 节。

5. 完成实验步骤 2.4.5 节。

6. 完成实验步骤 2.4.6 节。

7. 完成实验步骤 2.4.7 节。

aUSS数据库实验教

2.6 实验讨论与总结

【请将实验中遇到的问题描述、解决办法与思考讨论列在下面，并对本实验进行总结。】

aus数据库实验教